

CS342L – Advanced Database Management Systems Lab

ADVANCED SQL 2: PROCEDURES AND USER-DEFINED FUNCTIONS



Ghulam Ishaq Khan Institute of Engineering Sciences



Objective

The objective of this session is to write server-side PL/pgSQL code in PostgreSQL — user-defined functions and stored procedures, variables, control structures, and exception handling — using the dvdrental database from Lab 1.

Instructions

- Open the handout/lab manual in front of a computer in the lab during the session, with the dvdrental database already restored and available.
- Practice each new command by completing the examples and exercises.
- Turn in the answers for all the exercise problems as your lab report.
- When answering problems, indicate the commands you entered and the output displayed.

Contents

Introduction to PL/pgSQL	4
Why Functions and Procedures?	4
The PL/pgSQL Block Structure	4
Creating User-Defined Functions.....	4
CREATE FUNCTION Syntax	4
A Simple Scalar Function.....	5
RETURNS TABLE and Table-Valued Functions	5
Function Parameters: IN, OUT, INOUT, and Defaults	5
CREATE OR REPLACE and DROP FUNCTION.....	6
Variables and Assignment.....	7
DECLARE and the Assignment Operator	7
SELECT INTO.....	7
Control Structures.....	7
IF / ELSIF / ELSE	7
The CASE Statement	8
LOOP and EXIT WHEN	8
WHILE Loops	8
FOR Loops Over a Numeric Range	9
FOR Loops Over a Query	9
Exception Handling	10
BEGIN ... EXCEPTION ... WHEN	10
RAISE	10
Stored Procedures	10
Functions vs. Procedures	10
CREATE PROCEDURE and CALL	11
Transaction Control Inside Procedures.....	11
Putting It All Together.....	12
Exercise	13

Introduction to PL/pgSQL

So far every query in this course has been a single SQL statement sent from the client. PL/pgSQL is PostgreSQL's procedural extension to SQL — it lets you write server-side code with variables, loops, conditionals, and error handling, packaged as a function or procedure that lives inside the database itself.

Why Functions and Procedures?

- Reusability — write the logic once, call it from any query, application, or trigger.
- Performance — the whole block runs on the server in a single round trip, instead of many separate statements from the client.
- Encapsulation — callers use a simple name and parameter list without needing to know the underlying table structure.
- Consistency — business rules enforced in one place instead of duplicated across every application that touches the database.

The PL/pgSQL Block Structure

Every PL/pgSQL function or procedure body is built from the same block structure:

```
[ DECLARE
    -- variable declarations ]
BEGIN
    -- statements
[ EXCEPTION
    -- error handlers ]
END;
```

Code language: SQL (Structured Query Language) (sql)

Note: DECLARE and EXCEPTION are both optional. A minimal block is just BEGIN ... END;

Creating User-Defined Functions

CREATE FUNCTION Syntax

```
CREATE [OR REPLACE] FUNCTION function_name(parameter_list)
RETURNS return_type
LANGUAGE plpgsql
AS $$
DECLARE
    -- variables
BEGIN
    -- logic
    RETURN value;
END;
$$;
```

Code language: SQL (Structured Query Language) (sql)

Note: The \$\$... \$\$ pair is a dollar-quoted string — it lets the function body contain single quotes without needing to escape them. OR REPLACE lets you redefine a function without dropping it first.

A Simple Scalar Function

This function looks up one customer and returns their full name as a single text value:

```
CREATE OR REPLACE FUNCTION get_customer_full_name(p_customer_id INT)
RETURNS TEXT
LANGUAGE plpgsql
AS $$
DECLARE
    v_full_name TEXT;
BEGIN
    SELECT first_name || ' ' || last_name
    INTO v_full_name
    FROM customer
    WHERE customer_id = p_customer_id;

    RETURN v_full_name;
END;
$$;
```

Code language: SQL (Structured Query Language) (sql)

Call it exactly like a built-in function:

```
SELECT get_customer_full_name(1);
```

Code language: SQL (Structured Query Language) (sql)

RETURNS TABLE and Table-Valued Functions

A function can return a whole result set instead of a single value. RETURNS TABLE declares the output columns, and RETURN QUERY runs a SELECT whose rows become the function's output:

```
CREATE OR REPLACE FUNCTION get_films_by_category(p_category_name VARCHAR)
RETURNS TABLE(film_id INT, title VARCHAR, rental_rate NUMERIC)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT f.film_id, f.title, f.rental_rate
    FROM film f
    JOIN film_category fc ON fc.film_id = f.film_id
    JOIN category c      ON c.category_id = fc.category_id
    WHERE c.name = p_category_name
    ORDER BY f.title;
END;
$$;
```

Code language: SQL (Structured Query Language) (sql)

A table-valued function is queried using FROM, just like a real table:

```
SELECT * FROM get_films_by_category('Comedy');
```

Code language: SQL (Structured Query Language) (sql)

Function Parameters: IN, OUT, INOUT, and Defaults

Parameters are IN by default (the caller passes a value in). OUT parameters let a function return several named values at once without needing RETURNS TABLE:

```
CREATE OR REPLACE FUNCTION get_customer_stats(
    p_customer_id INT,
    OUT total_rentals INT,
    OUT total_paid NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    SELECT COUNT(*)
    INTO total_rentals
    FROM rental
    WHERE customer_id = p_customer_id;

    SELECT COALESCE(SUM(amount), 0)
    INTO total_paid
    FROM payment
    WHERE customer_id = p_customer_id;
END;
$$;

SELECT * FROM get_customer_stats(1);
```

Code language: SQL (Structured Query Language) (sql)

A parameter can also have a DEFAULT value, making it optional for the caller:

```
CREATE OR REPLACE FUNCTION films_over_price(p_min_rate NUMERIC DEFAULT 4.99)
RETURNS TABLE(title VARCHAR, rental_rate NUMERIC)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT f.title, f.rental_rate
    FROM film f
    WHERE f.rental_rate >= p_min_rate
    ORDER BY f.rental_rate DESC;
END;
$$;

SELECT * FROM films_over_price();           -- uses the default, 4.99
SELECT * FROM films_over_price(2.99);      -- overrides the default
```

Code language: SQL (Structured Query Language) (sql)

CREATE OR REPLACE and DROP FUNCTION

Use OR REPLACE to redefine a function in place, or DROP FUNCTION to remove it entirely:

```
DROP FUNCTION IF EXISTS films_over_price(NUMERIC);
```

Code language: SQL (Structured Query Language) (sql)

Note: PostgreSQL identifies a function by name AND parameter types, so two functions can share a name if their parameter lists differ — this is called overloading. DROP FUNCTION must include the parameter types to say which version to remove.

Variables and Assignment

DECLARE and the Assignment Operator

Variables are declared once in the DECLARE section, with a type and an optional default, then assigned with :=:

```
DECLARE
    v_count      INT := 0;
    v_title      VARCHAR(255);
    v_rate       NUMERIC(4,2) := 2.99;
BEGIN
    v_count := v_count + 1;
    v_title := 'ACADEMY DINOSAUR';
```

Code language: SQL (Structured Query Language) (sql)

SELECT INTO

SELECT INTO runs a query and stores its single-row result directly into one or more variables:

```
DECLARE
    v_title VARCHAR(255);
    v_length SMALLINT;
BEGIN
    SELECT title, length
    INTO v_title, v_length
    FROM film
    WHERE film_id = 1;
```

Code language: SQL (Structured Query Language) (sql)

Note: If the query matches no rows, the variables become NULL rather than raising an error — unless you use SELECT ... INTO STRICT, which raises NO_DATA_FOUND when nothing matches and TOO_MANY_ROWS when more than one row matches. STRICT is covered again under exception handling.

Control Structures

IF / ELSIF / ELSE

IF branches on a boolean condition, with optional ELSIF and ELSE arms:

```
CREATE OR REPLACE FUNCTION classify_film_length(p_length INT)
RETURNS TEXT
LANGUAGE plpgsql
AS $$
BEGIN
    IF p_length < 60 THEN
        RETURN 'Short';
```

```

    ELSIF p_length <= 120 THEN
        RETURN 'Medium';
    ELSE
        RETURN 'Long';
    END IF;
END;
$$;

SELECT title, classify_film_length(length) FROM film ORDER BY title LIMIT 5;

```

Code language: SQL (Structured Query Language) (sql)

The CASE Statement

PL/pgSQL also has a CASE statement (distinct from the SQL CASE expression from Lab 2) — it executes a block of statements rather than returning a value directly:

```

DECLARE
    v_tier TEXT;
BEGIN
    CASE
        WHEN p_rate < 2.99 THEN
            v_tier := 'Budget';
        WHEN p_rate < 4.99 THEN
            v_tier := 'Standard';
        ELSE
            v_tier := 'Premium';
    END CASE;

```

Code language: SQL (Structured Query Language) (sql)

LOOP and EXIT WHEN

LOOP repeats indefinitely until an EXIT (optionally guarded by EXIT WHEN) breaks out of it:

```

DECLARE
    v_i INT := 1;
BEGIN
    LOOP
        RAISE NOTICE 'i = %', v_i;
        v_i := v_i + 1;
        EXIT WHEN v_i > 5;
    END LOOP;
END;

```

Code language: SQL (Structured Query Language) (sql)

WHILE Loops

```

DECLARE
    v_i INT := 1;
BEGIN
    WHILE v_i <= 5 LOOP
        RAISE NOTICE 'i = %', v_i;
        v_i := v_i + 1;
    
```

```
END LOOP;
END;
```

Code language: SQL (Structured Query Language) (sql)

FOR Loops Over a Numeric Range

```
BEGIN
  FOR v_i IN 1..5 LOOP
    RAISE NOTICE 'i = %', v_i;
  END LOOP;
END;
```

Code language: SQL (Structured Query Language) (sql)

Note: The loop variable (v_i above) is declared implicitly by the FOR loop itself — it does not need a DECLARE entry.

FOR Loops Over a Query

A FOR loop can iterate over the rows of a query directly, which is one of PL/pgSQL's most useful features. This builds a comma-separated list of every film title a given actor appears in:

```
CREATE OR REPLACE FUNCTION films_for_actor(p_actor_id INT)
RETURNS TEXT
LANGUAGE plpgsql
AS $$
DECLARE
  v_rec  RECORD;
  v_list TEXT := '';
BEGIN
  FOR v_rec IN
    SELECT f.title
    FROM film f
    JOIN film_actor fa ON fa.film_id = f.film_id
    WHERE fa.actor_id = p_actor_id
    ORDER BY f.title
  LOOP
    IF v_list = '' THEN
      v_list := v_rec.title;
    ELSE
      v_list := v_list || ', ' || v_rec.title;
    END IF;
  END LOOP;
  RETURN v_list;
END;
$$;

SELECT films_for_actor(1);
```

Code language: SQL (Structured Query Language) (sql)

Note: RECORD is a generic row type — it takes on whatever columns the query inside the loop returns, so it doesn't need to match a specific table.

Exception Handling

BEGIN ... EXCEPTION ... WHEN

Wrapping statements in a block with an EXCEPTION section lets you catch specific error conditions instead of letting the whole function abort. SELECT ... INTO STRICT raises NO_DATA_FOUND when nothing matches:

```
CREATE OR REPLACE FUNCTION get_customer_email(p_customer_id INT)
RETURNS TEXT
LANGUAGE plpgsql
AS $$
DECLARE
    v_email TEXT;
BEGIN
    SELECT email
    INTO STRICT v_email
    FROM customer
    WHERE customer_id = p_customer_id;

    RETURN v_email;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE NOTICE 'No customer found with id %', p_customer_id;
        RETURN NULL;
    WHEN TOO_MANY_ROWS THEN
        RAISE NOTICE 'customer_id % is not unique – check the data', p_customer_id;
        RETURN NULL;
END;
$$;

SELECT get_customer_email(9999); -- no such customer: returns NULL and prints a notice
```

Code language: SQL (Structured Query Language) (sql)

RAISE

RAISE can log a message at several severity levels, or raise a genuine error that aborts the transaction:

```
RAISE NOTICE 'Just informational – execution continues';
RAISE WARNING 'Something looks off – execution still continues';
RAISE EXCEPTION 'Something is wrong – this aborts the current transaction';
```

Code language: SQL (Structured Query Language) (sql)

Note: WHEN OTHERS catches any exception not matched by an earlier WHEN clause — useful as a catch-all, but use it sparingly, since it can hide bugs you would otherwise want to see.

Stored Procedures

Functions vs. Procedures

Both package PL/pgSQL logic under a name, but they differ in a few important ways:

- A function is called as part of an expression (SELECT my_func(...)) and must RETURN a value (unless it returns void); a procedure is called on its own with CALL and does not use RETURN to produce a value.
- A procedure can contain COMMIT and ROLLBACK to control transactions from inside its own body; a function cannot.
- A procedure can return values only through OUT parameters, retrieved after CALL; a function can return a scalar, a row, or a whole table.

CREATE PROCEDURE and CALL

The syntax mirrors CREATE FUNCTION, but with no RETURNS clause, and it is invoked with CALL instead of SELECT:

```
CREATE OR REPLACE PROCEDURE process_rental_return(p_rental_id INT)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE rental
    SET return_date = NOW()
    WHERE rental_id = p_rental_id
    AND return_date IS NULL;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'rental_id % does not exist or was already returned', p_rental_id;
    END IF;
END;
$$;

CALL process_rental_return(5);
```

Code language: SQL (Structured Query Language) (sql)

Note: FOUND is a special boolean variable automatically set by the previous statement — true if it affected at least one row, false otherwise.

Transaction Control Inside Procedures

Because a procedure can COMMIT partway through, it can process a batch of work in stages that survive even if a later stage fails. This raises the rental_rate for every film in a category, committing after each film so earlier updates are not lost if a later one is rejected:

```
CREATE OR REPLACE PROCEDURE raise_category_prices(
    p_category_name VARCHAR,
    p_increase       NUMERIC
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_rec RECORD;
BEGIN
    FOR v_rec IN
        SELECT f.film_id
        FROM film f
        JOIN film_category fc ON fc.film_id = f.film_id
        JOIN category c      ON c.category_id = fc.category_id
```

```

        WHERE c.name = p_category_name
    LOOP
        UPDATE film
        SET rental_rate = rental_rate + p_increase
        WHERE film_id = v_rec.film_id;

        COMMIT;
    END LOOP;
END;
$$;

CALL raise_category_prices('Comedy', 0.50);

```

Code language: SQL (Structured Query Language) (sql)

Note: COMMIT and ROLLBACK are only allowed inside a procedure called directly with CALL from outside any other transaction block — not inside a function, and not inside a procedure called from within another procedure's own transaction.

Putting It All Together

A realistic procedure typically combines several of these pieces at once: parameters, variables, a lookup, a conditional check, an exception, and a multi-table write. This records a full rental transaction — it finds an available copy of the requested film, creates the rental, and charges the payment, all in one call:

```

CREATE OR REPLACE PROCEDURE rent_film(
    p_customer_id INT,
    p_film_id      INT,
    p_staff_id     INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_inventory_id INT;
    v_rental_rate  NUMERIC;
BEGIN
    -- find an inventory copy of this film that is not currently on rent
    SELECT i.inventory_id
    INTO v_inventory_id
    FROM inventory i
    WHERE i.film_id = p_film_id
    AND NOT EXISTS (
        SELECT 1 FROM rental r
        WHERE r.inventory_id = i.inventory_id
        AND r.return_date IS NULL
    )
    LIMIT 1;

    IF v_inventory_id IS NULL THEN
        RAISE EXCEPTION 'no available copies of film_id %', p_film_id;
    END IF;

    SELECT rental_rate INTO v_rental_rate FROM film WHERE film_id = p_film_id;

```

```

INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id)
VALUES (NOW(), v_inventory_id, p_customer_id, p_staff_id);

INSERT INTO payment (customer_id, staff_id, rental_id, amount, payment_date)
VALUES (
    p_customer_id, p_staff_id,
    (SELECT rental_id FROM rental WHERE inventory_id = v_inventory_id ORDER BY rental_id
DESC LIMIT 1),
    v_rental_rate, NOW()
);
END;
$$;

CALL rent_film(1, 1, 1);

```

Code language: SQL (Structured Query Language) (sql)

Exercise

1. Write a scalar function `total_films_in_category(p_category_name VARCHAR)` that returns the number of films in the given category.
2. Write a table-valued function `top_customers(p_limit INT DEFAULT 5)` that returns the top `p_limit` customers by total payments, with columns `customer_id`, `full_name`, and `total_paid`.
3. Write a function `classify_customer(p_customer_id INT)` that uses IF/ELSIF to return 'Frequent' for customers with more than 30 rentals, 'Regular' for 10–30, and 'Occasional' for fewer than 10.
4. Write a procedure `transfer_inventory(p_inventory_id INT, p_new_store_id INT)` that moves one inventory item to a different store, raising an exception if the `inventory_id` does not exist.
5. Write a function `customer_emails_in_city(p_city VARCHAR)` that uses a FOR loop over a query to build and return a comma-separated string of every customer's email address in the given city.
6. Write a function `get_film_title(p_film_id INT)` that uses SELECT INTO STRICT and returns the film's title, or returns NULL and raises a notice if no film with that ID exists.
7. Write a procedure `apply_late_fee(p_rental_id INT, p_fee NUMERIC)` that inserts a payment row for the given rental's customer and staff as an extra fee, raising an exception if the `rental_id` does not exist.
8. Write a procedure `deactivate_inactive_customers(p_cutoff_date DATE)` that loops through every customer whose most recent rental is before `p_cutoff_date`, sets their active flag to 0, and commits after each update.